

the chapter about *asynchronous programming* to emphasize core language features—e.g., `await`, `async dev`, `async for`, and `async with`, and show how they are used with *asyncio* and other frameworks.

Part V, “Metaprogramming”

This part starts with a review of techniques for building classes with attributes created dynamically to handle semi-structured data, such as JSON datasets. Next, we cover the familiar properties mechanism, before diving into how object attribute access works at a lower level in Python using descriptors. The relationship among functions, methods, and descriptors is explained. Throughout **Part V**, the step-by-step implementation of a field validation library uncovers subtle issues that lead to the advanced tools of the final chapter: class decorators and metaclasses.

Hands-On Approach

Often we’ll use the interactive Python console to explore the language and libraries. I feel it is important to emphasize the power of this learning tool, particularly for those readers who’ve had more experience with static, compiled languages that don’t provide a read-eval-print loop (REPL).

One of the standard Python testing packages, **doctest**, works by simulating console sessions and verifying that the expressions evaluate to the responses shown. I used **doctest** to check most of the code in this book, including the console listings. You don’t need to use or even know about **doctest** to follow along: the key feature of doctests is that they look like transcripts of interactive Python console sessions, so you can easily try out the demonstrations yourself.

Sometimes I will explain what we want to accomplish by showing a doctest before the code that makes it pass. Firmly establishing what is to be done before thinking about how to do it helps focus our coding effort. Writing tests first is the basis of test-driven development (TDD), and I’ve also found it helpful when teaching. If you are unfamiliar with **doctest**, take a look at its **documentation** and this book’s **example code repository**.

I also wrote unit tests for some of the larger examples using *pytest*—which I find easier to use and more powerful than the *unittest* module in the standard library. You’ll find that you can verify the correctness of most of the code in the book by typing `python3 -m doctest example_script.py` or `pytest` in the command shell of your OS. The *pytest.ini* configuration at the root of the **example code repository** ensures that doctests are collected and executed by the `pytest` command.